

LABORATORIO 11

¿Quién autorizó ese campo nuevo?

*Schema Registry, la única pieza del ecosistema cuyo trabajo es decir que no.
Guía para hacerlo por tu cuenta, paso a paso*

Confluent Platform 8.2.0 · 3 brokers KRaft · Schema Registry · Avro

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Éste es el laboratorio donde Kafka deja de confiar en la buena voluntad. Hasta ahora, todo lo que escribiste en un tópico entró. **Kafka no miró adentro ni una sola vez.**

Hoy vas a poner en el camino la única pieza del ecosistema cuyo trabajo es decir que no. Y la vas a ver decir que no **y decir que sí**, con el mismo comando, porque una compuerta que rechaza todo no sirve de nada.

LO QUE ESTE LABORATORIO DEMUESTRA

El contrato de datos no es un documento. Es una compuerta, y se demuestra con lo que rechaza.

Tres partes, y las tres se ven en pantalla.

La parte	Dónde se comprueba
Es una compuerta	Paso 3. Hay un momento exacto y una respuesta exacta en que el cambio no entra
Y no rechaza en silencio	Paso 3. Dice qué campo, por qué, y contra qué versión lo comparó
No hay puerta trasera	Paso 4. Al que escribe con <code>curl</code> y al que escribe con un productor le pasa lo mismo

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_4/lab-11-schema-registry</code>
Los puertos 9092, 9093, 9094, 8081 y 8090 libres	El paso 1 te avisa si no lo están
Nada más	Todo el laboratorio va por <code>curl</code> , que ya tienes. No hace falta instalar nada

SOBRE POR QUÉ ESTA GUÍA USA `CURL` Y NO LOS ENVOLTORIOS DEL CURSO

El laboratorio trae unos scripts en `schema-cli/` que hacen lo mismo en una línea. **Esta guía no los usa en el recorrido**, y conviene que sepas por qué antes de que los descubras solo.

Schema Registry es una API REST y nada más. El `curl` que vas a escribir aquí es **exactamente el que vas a escribir en el servidor de SUNAT**, donde no hay envoltorios de curso.

Y hay una razón práctica. **Esos envoltorios necesitan `python3`, que Git Bash para Windows no trae**, y ningún README lo declara. En el *para profundizar* está medido qué pasa exactamente cuando falta, porque uno de ellos falla de una forma que engaña.

El caso

Un martes, un desarrollador del equipo de *fulfillment* de **NovaTech Logistics** agrega un campo al pedido. Necesita guardar el medio de pago, y agregar un campo es la cosa más inocente que se le puede hacer a un registro.

Lo prueba en su máquina, funciona, lo sube. El productor no falla. El tópico sigue recibiendo. Los tableros siguen dibujando. **Nadie se entera de nada, porque no hay nada de qué enterarse.**

El jueves, el equipo de *analytics* corre el reproceso del mes para el cierre. Y el proceso se cae. No en el primer mensaje. Se cae **en el mensaje cuarenta mil y algo**, que es donde el histórico viejo se junta con lo que se escribió desde el martes.

Nadie rompió nada a propósito. El del martes agregó un campo. El de *analytics* no tocó una línea de código. **Y el cierre no salió.**

LA METÁFORA · HOY ENTRAN EL FORMATO DE LA COMANDA Y EL TALONARIO

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones, las **libretas de respaldo** son las réplicas y el **cocinero** es el consumidor.

Hoy entran dos piezas. **El formato de la comanda**, o sea qué casillas trae y cuáles son obligatorias, que es el **schema**. Y **el talonario oficial del local**, el único lugar donde está definido cómo es una comanda válida y quién decide si se puede cambiar, que es el **Schema Registry**.

Hasta ahora, en este curso, el salón y la cocina se entendían porque sí. Nadie había escrito nunca en qué casilla va cada cosa. Funcionaba porque el local es chico y todos se conocen.

Ahora imagina que alguien del salón manda a imprimir un talonario nuevo. Le agrega una casilla, que dice **número de tarjeta**, y la deja **obligatoria**. Una comanda sin esa casilla llena no es una comanda válida. Los mozos empiezan a usar el talonario nuevo esa misma tarde. Nadie avisó a la cocina, porque para el salón no cambió nada.

Y entonces la cocina, que viene con retraso, sigue sacando comandas del pincho del turno anterior. Comandas escritas con el talonario **viejo**. El cocinero, que ya aprendió el formato nuevo, llega a la casilla del número de tarjeta y **no está**. No está en blanco. No existe. Y nadie le dijo nunca qué hacer cuando no está.

La cocina se detiene. Y no se detuvo por culpa de la comanda que tiene en la mano, que está perfecta. **Se detuvo por culpa de un talonario que se cambió sin preguntar.**

LA REGLA QUE SALE DE AHÍ

El talonario no se cambia sin preguntarle a la cocina **si va a poder seguir leyendo lo que ya está escrito**. Eso es exactamente lo que hace Schema Registry, y para lo que hoy nos importa es lo único que hace.

Las tres piezas, con sus nombres

CONCEPTO · SCHEMA

La declaración de qué campos tiene un mensaje, de qué tipo es cada uno, y cuáles pueden faltar. Aquí está escrito en **Avro**, que es un archivo `.avsc` con formato JSON. **Es el formato de la comanda.**

CONCEPTO · SCHEMA REGISTRY

Un servicio **aparte del clúster**, aquí el contenedor `schema-registry`, que responde en `http://localhost:8081`. No es parte del broker. Es una pieza más del ecosistema Confluent, con su propia API REST. **Es el talonario oficial.**

CONCEPTO · SUBJECT

El nombre bajo el que se guarda **la historia** de un schema. Por convención es `<tópico>-value` para el valor y `<tópico>-key` para la clave. El tuyo va a ser `novatech.lab10.pedidos-value`. **Un subject no tiene un schema. Tiene una lista de versiones, y esa lista solo crece.**

CONCEPTO · COMPATIBILIDAD BACKWARD

El modo con el que viene configurado este clúster. Dice que **un lector que ya tiene el schema nuevo tiene que poder leer los datos escritos con el schema viejo**. Es el cocinero de la metáfora, que aprendió el talonario nuevo y sigue sacando comandas viejas del pincho.

La palabra *backward* confunde a todo el mundo la primera vez, así que conviene fijar quién es quién.

Quién	Qué tiene	Qué se le exige
El lector , o sea el consumidor	El schema nuevo	Que no se caiga
El dato	Escrito con el schema viejo	Nada. Ya está escrito y no se puede cambiar

Y AQUÍ ESTÁ LA CONSECUENCIA QUE NINGUNA DEFINICIÓN DE UNA LÍNEA DICE

Bajo `BACKWARD`, **agregar un campo obligatorio es un cambio incompatible.**

Agregar campos no está mal. Lo que pasa es que el lector nuevo, al toparse con un dato viejo, va a buscar ese campo y no lo va a encontrar. **Y nadie le dijo qué poner en su lugar.**

CONCEPTO · VALOR POR DEFECTO, O DEFAULT

Lo que el lector debe asumir cuando el campo no viene en el dato. **Un campo con default se puede agregar sin romper a nadie**, porque el lector nuevo lee un dato viejo, no encuentra el campo y usa el *default*. Un campo sin *default* no tiene salida.

El campo que el desarrollador del martes agregó es `tarjeta_credito`, obligatorio y sin *default*. **Es exactamente el caso que no tiene salida**, y es el que vas a intentar registrar en el paso 3.

Levantar el clúster y mirar el talonario vacío

Qué vamos a hacer

Encender el laboratorio y comprobar, antes de nada, que **el Registry está vacío**. Es un dato que importa, porque explica de dónde sale todo lo demás.

El comando

```
cd Capitulo_4/lab-11-schema-registry
chmod +x bin/*.sh schema-cli/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

chmod +x — permiso de ejecución para las **tres** carpetas de scripts de este lab

bin/start-lab.sh — levanta los tres brokers, el Schema Registry y Kafbat UI, y crea dos tópicos. **Y borra los volúmenes**, que es lo que deja el contador de schemas en cero

EL FINAL DE LO QUE VAS A VER · RECORTADO

```
Created topic novatech.lab10.pedidos.
  ✓ Tópico novatech.lab10.pedidos creado (12 particiones, RF=3)
Created topic novatech.lab10.clientes.
  ✓ Tópico novatech.lab10.clientes creado (3 particiones, RF=3)
✓ Tópicos del Lab 11 inicializados
```

CLÚSTER NOVATECH LAB 11 OPERATIVO

Componentes:

Broker 1:	localhost:9092
Broker 2:	localhost:9093
Broker 3:	localhost:9094
Kafbat UI:	http://localhost:8090
Schema Registry:	http://localhost:8081 ← API REST

Tópicos del Lab 11:

novatech.lab10.pedidos	- 12 particiones, datos Avro
novatech.lab10.clientes	- 3 particiones, datos Avro

DOS COSAS DEL ARRANQUE QUE SORPRENDEN

Los tópicos se llaman `lab10` y estás en el laboratorio 11. No es un error. Este lab continúa el conjunto de datos del anterior y conserva sus nombres a propósito, para que se vea que el contrato se le pone a un tópico que ya existía.

Y `novatech.lab10.pedidos` tiene 12 particiones, no 3 como los de los labs anteriores. Por eso, cuando cuentes mensajes, vas a tener que sumar doce líneas. El paso 4 lo resuelve con un `awk`.

El talonario, antes de que nadie escriba en él

```
curl -s http://localhost:8081/subjects
```

`curl` — el cliente HTTP de línea de comandos. Aquí no hay ningún `docker` adelante, porque el Registry publica su puerto 8081 hacia tu máquina y le hablas directo

`-s` — modo silencioso. Sin esto `curl` dibuja una barra de progreso encima de la respuesta

`/subjects` — la lista de contratos registrados

LO QUE VAS A VER

```
[]
```

Una lista vacía. Un clúster recién levantado no trae ni un contrato. Todo lo que haya en el Registry lo puso alguien, y hoy ese alguien vas a ser tú.

VAS BIEN SI...

El `curl` contestó `[]`, con los corchetes y nada dentro. Y hay cinco contenedores arriba:

```
docker ps --filter "label=com.docker.compose.project=novatech-lab11" \
  --format 'table {{.Names}}\t{{.Status}}'
```

Tienen que estar los tres brokers, `schema-registry` y `kafbat-ui`.

SI EL `CURL` DICE `FAILED TO CONNECT TO LOCALHOST PORT 8081`

El Registry no está arriba, o todavía está arrancando. Tarda más que los brokers porque espera a que ellos estén listos.

Dale medio minuto y repite. Si sigue, mira sus registros:

```
docker logs --tail 30 schema-registry
```

SI EL `CURL` TE DEVUELVE UNA LISTA CON COSAS DENTRO

Ya corriste este laboratorio antes y no volviste a levantarlo desde cero. **No es un problema para el recorrido**, pero los números del paso 2 te van a salir distintos. Si quieres las salidas exactas de esta guía, corre `bin/reset-lab.sh` y luego `bin/start-lab.sh`.

SI EL PUERTO 8081 ESTÁ OCUPADO

En este curso lo usa también la aplicación de Spring del laboratorio 09. Comprueba que no quedó corriendo, y si la levantaste tú a mano, bájala con `Ctrl+C` en su terminal.

Escribir el contrato

Qué vamos a hacer

Mirar el contrato que van a firmar los tres equipos, y registrarlo. Es el pedido tal como lo conocen hoy, con seis campos.

Primero, mirarlo

```
cat infra/schemas/pedido.avsc
```

LO QUE VAS A VER

```
{
  "type": "record",
  "name": "Pedido",
  "namespace": "com.novatech.lab10",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "cliente_id", "type": "int"},
    {"name": "producto", "type": "string"},
    {"name": "cantidad", "type": "int"},
    {"name": "monto", "type": "double"},
    {"name": "estado", "type": "string"}
  ]
}
```

Seis campos, cada uno con su tipo. **Ninguno tiene la palabra `default`**. Los seis son obligatorios, así que un mensaje al que le falte cualquiera de ellos no es un pedido válido.

Guarda eso, porque es la mitad de la explicación del paso 3.

Ahora se registra

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"$(tr -d '\\n' < infra/schemas/pedido.avsc | sed
's/"/\\\\"/g')\"}" \
  http://localhost:8081/subjects/novatech.lab10.pedidos-value/versions
```

Es el comando más incómodo del laboratorio, y conviene desarmarlo entero, porque **los mismos pedazos se repiten en los pasos 3 y 5**.

-s — silencia la barra de progreso, que en una respuesta corta estorba más que ayuda

-w "\nHTTP %{http_code}\n" — **imprime el código HTTP al final**. Sin esto no lo ves, y en este laboratorio el código es la respuesta

-X POST — registrar es escribir. Un `GET` sobre la misma URL solo lista las versiones

-H "Content-Type: ...v1+json" — el tipo de contenido propio del Registry. **Sin esta cabecera contesta 415**

--data "{ \"schema\": \"...\" }" — el cuerpo. **El schema no viaja como JSON. Viaja como una cadena de texto dentro de un JSON**, y de ahí sale todo lo feo de esta línea

tr -d '\n' < ...avsc — aplasta el archivo a una sola línea. **Una cadena JSON no puede llevar saltos de línea**

sed 's/"/\\\"/g' — escapa cada comilla del schema, para que no cierre la cadena que lo contiene

/subjects/<subject>/versions — la URL. El subject es `novatech.lab10.pedidos-value`, y el **-value** **no es decorativo**. Dice que este contrato es para el valor del mensaje, no para su clave

LO QUE VAS A VER · RECORTADO, PORQUE DEVUELVE EL SCHEMA ENTERO

```
{ "id":1, "version":1, "guid":"f7c3dc5b-9a9f-1792-4c66-77095fdbd42d", "schemaType":"AVRO", "schema":{"type":"record", "name":"Pedido", ...}}
HTTP 200
```

Cómo se lee

Son dos números distintos, y conviene no confundirlos nunca.

Campo	Qué dice
<code>version: 1</code>	La versión dentro de este subject . Es la que le importa a la gente, o sea «el contrato de pedidos va en la v1»
<code>id: 1</code>	El identificador global del Registry , único en todo el clúster. Es el que viaja pegado a cada mensaje . Hoy coinciden porque es el primer schema del laboratorio. En un clúster con veinte tópicos no coinciden nunca
<code>schemaType: AVRO</code>	El Registry también acepta JSON Schema y Protobuf. Aquí es Avro

EL ID ES LA PIEZA QUE HACE QUE AVRO PESE TAN POCO

Cada mensaje que se escriba desde ahora **no lleva el schema adentro. Lleva ese número, en cuatro bytes**. El que lee usa el número para pedirle el schema al Registry una sola vez. Los nombres de los campos, o sea `cliente_id`, `producto` y `monto`, no se repiten en cada mensaje como sí pasa en JSON.

VAS BIEN SI...

`HTTP 200`, y la respuesta trae `version` e `id`. **El contrato está firmado.** Desde este momento el tópico tiene reglas.

SI LOS DOS NÚMEROS NO TE SALEN EN 1

No está roto. El `id` es un contador global del Registry que nunca reutiliza valores. Si ya corriste este laboratorio antes sin volver a levantar el clúster, tu `id` será 3, o 7.

El único que vuelve a 1 es el `version`, y solo si el subject se borró permanentemente. Para tener la salida exacta de esta guía, arranca con `bin/start-lab.sh`, que borra los volúmenes y con ellos el tópico `_schemas` donde vive ese contador.

SI CONTESTA `HTTP 415`

Falta la cabecera `Content-Type`, o la escribiste incompleta. Tiene que ser `application/vnd.schemaregistry.v1+json` entera. **El 415 quiere decir «no hablo ese formato»**, y lo dice el Registry.

SI CONTESTA `HTTP 422` O SE QUEJA DEL CUERPO

El escape se rompió. Casi siempre es que copiaste el comando partido en varias líneas y se perdió una barra invertida del final. **Cópialo entero de nuevo**, y comprueba primero qué está generando el escape:

```
tr -d '\n' < infra/schemas/pedido.avsc | sed 's/"\\/"/g'
```

El cambio que parece inocente

Qué vamos a hacer

Exactamente lo que hizo el desarrollador del martes. **Agregar un campo.**

Primero, la única diferencia

```
diff infra/schemas/pedido.avsc infra/schemas/pedido-v3-incompatible.avsc
```

diff — compara dos archivos línea por línea. **No toca nada ni habla con el clúster**

el primer archivo — el contrato **vigente**, el que ya está registrado

el segundo archivo — el contrato **propuesto**

LO QUE VAS A VER

```
11c11,12
<     {"name": "estado", "type": "string"}
---
>     {"name": "estado", "type": "string"},
>     {"name": "tarjeta_credito", "type": "string"}
```

El `11c11,12` dice que la línea 11 del primero cambia por las líneas 11 y 12 del segundo. Lo que va después del `<` es lo viejo y lo que va después del `>` es lo nuevo.

El cambio es una sola línea. Un campo llamado `tarjeta_credito`, de tipo `string`. Sin *default*.

DETENTE AQUÍ Y CONTESTA ANTES DE SEGUIR

Es un campo nuevo, al final, que no toca ninguno de los seis que ya estaban. No renombra nada, no borra nada, no cambia ningún tipo. **Para casi cualquier persona esto es el cambio más seguro que existe.** ¿Va a entrar?

Preguntar sin escribir

Esto importa en la vida real. **Existe una forma de preguntar sin registrar nada.**

El comando es el del paso 2 con **una sola diferencia**, que es la URL. En vez de

`/subjects/.../versions`, que registra, va a `/compatibility/...`, que solo pregunta.

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"\$(tr -d '\\n' < infra/schemas/pedido-v3-incompatible.avsc | sed
's"/\\\"/g')\\\"}\" \
  http://localhost:8081/compatibility/subjects/novatech.lab10.pedidos-
value/versions/latest
```

`/compatibility/subjects/<subject>/versions/latest` — es de solo consulta. Pregunta y no registra nada, pase lo que pase. El `latest` dice contra qué versión comparar

todo lo demás — idéntico al paso 2. El mismo escape, la misma cabecera, el mismo `-w`

LO QUE VAS A VER

```
{"is_compatible":false}
HTTP 200
```

FÍJATE EN LA TRAMPA DEL `HTTP 200`

El código dice que **la pregunta se respondió**, no que la respuesta sea que sí. Un *pipeline* que solo mire el código HTTP de este comando va a creer que todo está bien y va a desplegar igual. **Lo que hay que leer es el cuerpo.**

Éste es el comando que un equipo pone en su integración continua. **Se pregunta antes de desplegar, no después.**

Y ahora sí, se intenta registrar

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"\$(tr -d '\\n' < infra/schemas/pedido-v3-incompatible.avsc | sed
's"/\\\"/g')\\\"}\" \
  http://localhost:8081/subjects/novatech.lab10.pedidos-value/versions \
  | fold -s -w 88
```

`| fold -s -w 88` — corta la respuesta en líneas de 88 caracteres. El `-s` corta en espacios y no a mitad de palabra. **Sin esto la respuesta es una sola línea larguísima** y no hay forma de leerla

LO QUE VAS A VER · EL `OLDSHEMA` DEL MEDIO VA RECORTADO

```
{
  "error_code": 40901,
  "message": "Schema being registered is incompatible with an earlier schema for subject \"novatech.lab10.pedidos-value\", details:
  [{
    \"errorType\": \"READER_FIELD_MISSING_DEFAULT_VALUE\",
    \"description\": \"The field 'tarjeta_credito' at path '/fields/6' in the new schema has no default value and is missing in the old schema\",
    \"additionalInfo\": \"tarjeta_credito\",
    \"oldSchemaVersion\": 1,
    \"oldSchema\": \"{\\\"type\\\":\\\"record\\\",\\\"name\\\":\\\"Pedido\\\",...}\\\"\",
    \"validateFields\": \"false\",
    \"compatibility\": \"BACKWARD\"}]\"
}
```

HTTP 409

Cómo se lee

Esa pared de texto es el laboratorio entero. Léela despacio, porque **el Registry está explicando su propia decisión**, y hay cuatro datos adentro.

Lo que dice	Qué significa
<code>errorType:</code> <code>READER_FIELD_MISSING_DEFAULT_VALUE</code>	El nombre del motivo. Traducido, al que lee le falta un campo y no tiene qué poner en su lugar . Es el cocinero de la metáfora, exacto
<code>The field 'tarjeta_credito' ... has no default value and is missing in the old schema</code>	Nombra el campo culpable . No dice «hay una incompatibilidad». Dice cuál, y dice las dos condiciones que la causan
<code>oldSchemaVersion: 1</code>	Contra qué versión comparó. Bajo <code>BACKWARD</code> compara contra la última , no contra todas
<code>compatibility: 'BACKWARD'</code>	Con qué regla juzgó . El mismo cambio, bajo otro modo, tendría otro veredicto. La compuerta no es una ley universal. Es una política que alguien configuró, y el paso 6 va a buscar a ese alguien

Y abajo del todo, `HTTP 409`. No es un `400 Bad Request`. **El schema no está mal escrito**, porque es Avro perfectamente válido. Está en conflicto con algo que ya existe. El `error_code: 40901` del cuerpo es el subcódigo propio del Registry dentro de ese 409.

COMPARA ESTE 409 CON EL 200 DE HACE UN MINUTO

El mismo archivo, la misma cabecera, el mismo escape. **Lo único que cambió fue la URL**. Preguntar siempre devuelve 200. Registrar es el que se atreve a devolver 409.

Y AQUÍ ESTÁ LA LECCIÓN, QUE NO SE DEDUCE SOLA

El desarrollador del martes no habría llegado al jueves. Su despliegue se cae el martes, en su propia pantalla, con el nombre del campo que lo rompió. Eso es todo lo que compra Schema Registry. **Adelantar el error al momento y a la persona que lo puede arreglar.**

VAS BIEN SI...

La consulta te dio `false` con `HTTP 200`, y el registro te dio `HTTP 409` con el nombre `tarjeta_credito` dentro del mensaje.

Anota una respuesta. ¿Por qué la consulta devolvió 200 si la respuesta era que no?

SI EL REGISTRO TE DEVUELVE `HTTP 200` EN VEZ DE 409

Pasa una de dos cosas.

Una. Te saltaste el paso 2 y no hay contrato vigente contra el que comparar. **El primer schema de un subject siempre entra**, porque no hay nada con lo que pueda chocar. Comprueba con `curl -s http://localhost:8081/subjects`.

Dos. El modo de compatibilidad no es `BACKWARD`. Míralo con el comando del paso 6.

SI LA RESPUESTA SALE COMO UNA SOLA LÍNEA ILEGIBLE

Se te olvidó el `| fold -s -w 88` del final. Vuelve a correr el comando con él. **No cambia la respuesta**, solo la parte en líneas.

Y no hay puerta trasera

Qué vamos a hacer

Aquí siempre aparece la misma objeción, y es buena. *Eso es porque usaste el comando del Registry. Yo escribo con un productor de Kafka, y el broker no sabe nada de contratos.*

Es cierto que el broker no sabe nada. **El contrato no lo aplica el broker. Lo aplica la biblioteca del cliente, antes de que el mensaje salga.**

Y la forma de demostrarlo es la que vale para cualquier compuerta. **Hacerla decir que no, y después hacerla decir que sí, con el mismo comando.**

Primero, cuántos mensajes hay

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --time -1 \
  | awk -F: '{s+= $3} END {print s" mensajes"}'
```

`docker exec kafka-broker-1` — ejecuta dentro del contenedor del broker 1, que es donde vive la herramienta. Sin `-i` ni `-t`, porque no le vas a escribir nada

`kafka-get-offsets` — pregunta por los offsets de un tópico. Solo consulta

`--time -1` — el offset **más nuevo**. Con `-2` pedirías el más antiguo

`| awk -F: '{s+= $3} ...'` — la salida trae una línea por partición, con formato

`tópico:partición:offset`. El `awk` corta por los dos puntos, suma la tercera columna y deja un solo número. **Este tópico tiene 12 particiones y sumarlas a ojo no funciona**

LO QUE VAS A VER

```
0 mensajes
```

Ahora el productor, con el contrato que el Registry rechazó

```
echo '{"id":99,"cliente_id":1001,"producto":"Caja
premium","cantidad":1,"monto":25000.00,"estado":"pendiente","tarjeta_credito":"4111-1111-
1111-1111"}' \
| MSYS_NO_PATHCONV=1 docker exec -i \
  -e SCHEMA_REGISTRY_LOG4J_OPTS="-Dlog4j2.configurationFile=/etc/cp-base-
java/log4j2.yaml" \
  schema-registry kafka-avro-console-producer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos \
  --reader-property schema.registry.url=http://schema-registry:8081 \
  --reader-property value.schema="$(tr -d '\n' < infra/schemas/pedido-v3-
incompatible.avsc)" \
  2>&1 | grep -oE "^org[^:]*Exception"
```

Es el comando más largo del laboratorio, y cada pedazo hace falta.

`echo '{...}'` — el mensaje, con los siete campos del contrato nuevo. El productor de consola lee de la entrada estándar

`MSYS_NO_PATHCONV=1` — solo para Git Bash en Windows. Sin esto, Git Bash ve el `/etc/cp-base-java/...` de la línea siguiente, cree que es una ruta y la traduce a una ruta de Windows **antes de que Docker la vea**. El contenedor recibe entonces algo como `C:/Program Files/Git/etc/...`, que dentro no existe. Fuera de Windows nadie lee esta variable y no molesta

`docker exec -i` — el `-i` mantiene abierta la entrada estándar, que es por donde entra el `echo`. **Aquí no va `-t`**, porque no hay una persona tecleando

`-e SCHEMA_REGISTRY_LOG4J_OPTS=...` — silencia el volcado de configuración que la imagen imprime por defecto. Sin esto la pantalla se llena con cuarenta líneas de `INFO` antes del error

`schema-registry kafka-avro-console-producer` — corre dentro del contenedor **del Registry**, no del broker. Las herramientas de Avro vienen en esa imagen y no en la de Kafka

`--reader-property schema.registry.url=...` — a qué Registry preguntarle. Sin esto el productor no sabe traducir a Avro y falla

`--reader-property value.schema=...` — el contrato con el que este productor dice que va a escribir

`$(tr -d '\n' < ...)` — aplasta el `.avsc` a una sola línea, porque el flag no acepta saltos de línea

`2>&1 | grep -oE "^org[^:]*Exception"` — el error viene con veinte líneas de traza de Java. Esto deja **solo el nombre de la excepción**. El motivo detallado ya lo leíste en el paso 3, y es el mismo

LO QUE VAS A VER

```
org.apache.kafka.common.errors.SerializationException
```

Dice *Serialization*, y no *Authorization* ni *Broker*. **El mensaje nunca llegó a la red**. Se cayó al convertirlo a bytes, dentro del proceso del productor, porque el paso de convertir a Avro incluye ir a preguntarle al Registry, y el Registry dijo que no.

Y el número que cierra el argumento

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --time -1 \
  | awk -F: '{s+= $3} END {print s " mensajes"}'
```

LO QUE VAS A VER

0 mensajes

Sigue en cero. No entró un mensaje a medias, ni un mensaje corrupto que haya que limpiar después. No entró nada.

Y ahora la mitad que hace que esto sea una demostración

El mismo comando, con lo único distinto siendo **el archivo del contrato**.

```
echo '{"id":1,"cliente_id":1001,"producto":"Caja
premium","cantidad":10,"monto":25000.00,"estado":"pendiente"}' \
| MSYS_NO_PATHCONV=1 docker exec -i \
  -e SCHEMA_REGISTRY_LOG4J_OPTS="-Dlog4j2.configurationFile=/etc/cp-base-
java/log4j2.yaml" \
  schema-registry kafka-avro-console-producer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos \
  --reader-property schema.registry.url=http://schema-registry:8081 \
  --reader-property value.schema="$(tr -d '\n' < infra/schemas/pedido.avsc)" \
  2>&1 | grep -oE "^org[^:]*Exception"
```

LO QUE VAS A VER

(nada, ni una línea)

Nada. El `grep` no encontró ninguna excepción que mostrar, porque no hubo ninguna. Y el tópico:

EL MISMO `KAFKA-GET-OFFSETS` DE ANTES

1 mensajes

Cómo se lee

Mismo productor, mismo broker, mismo tópico, mismo mensaje de seis campos. Lo único que cambió es **con qué contrato dijo que escribía**. Uno pasó y el otro no.

Momento	Contrato declarado	Resultado	Mensajes en el tópic
Antes de intentar nada	ninguno	ninguno	0
Intento 1	<code>pedido-v3-incompatible.avsc</code>	<code>SerializationException</code>	0
Intento 2	<code>pedido.avsc</code>	Sin error	1

ÉSA ES LA PRUEBA DE QUE LA COMPUERTA ES UNA COMPUERTA

Una que rechaza todo no sirve de nada. **Hay que verla dejar pasar.** Y ésta la viste hacer las dos cosas, con el mismo comando y en el mismo minuto.

LO LOGRASTE SI...

Tu tabla dice **0, 0, 1. Anota una respuesta.** Si el broker no sabe nada de contratos, ¿quién rechazó el primer mensaje, y en qué máquina se ejecutó ese rechazo?

SI EL COMANDO SE QUEJA DE UNA RUTA RARÍSIMA CON `C:/PROGRAM FILES/GIT/...`

Se te olvidó el `MSYS_NO_PATHCONV=1` del principio, o lo pusiste después del `docker`. **Va delante de `docker exec`, no en otro sitio.** Es la traducción de rutas de Git Bash actuando donde no debe.

SI SALEN CUARENTA LÍNEAS DE `INFO` ANTES DEL ERROR

Falta el `-e SCHEMA_REGISTRY_LOG4J_OPTS=...`. **El resultado es el mismo**, solo que enterrado. Busca la línea que empieza con `org.apache.kafka`.

SI EL SEGUNDO INTENTO TAMBIÉN FALLA

Comprueba que estás usando `infra/schemas/pedido.avsc`, sin el `-v3-incompatible`. Es el error más fácil de cometer, porque los dos comandos son idénticos salvo esa palabra.

Y comprueba que el mensaje del `echo` tiene **seis campos y no siete**. Si le dejaste el `tarjeta_credito`, el productor va a fallar por otra razón, que es que el dato no calza con el contrato que declaró.

SI EL CONTADOR NO SUBE A 1

El `awk` suma las 12 particiones. Si te da otro número mayor, es que ya habías producido antes en este tópico, y no es un problema. **Lo que importa no es el número exacto, sino que subió respecto del intento anterior.** Vuelve a correr el `kafka-get-offsets` antes y después para verlo.

El cambio que sí pasa

Qué vamos a hacer

Si el laboratorio terminara en el paso 4, te llevarías una conclusión falsa y peligrosa, que es *con Schema Registry no se pueden agregar campos. Se pueden*. Hay exactamente una condición.

La otra propuesta

```
diff infra/schemas/pedido.avsc infra/schemas/pedido-v2-compatible.avsc
```

LO QUE VAS A VER

```
11c11,12
<      {"name": "estado", "type": "string"}
---
>      {"name": "estado", "type": "string"},
>      {"name": "prioridad", "type": ["null", "string"], "default": null}
```

Otra vez **una sola línea**, y un campo nuevo al final. La misma forma de cambio que el paso 3. Y dos diferencias que lo cambian todo.

Lo que trae	Qué significa
<code>"type": ["null", "string"]</code>	Un tipo unión . Este campo puede ser un texto o puede ser nulo. En Avro, un campo que admite nulo se declara así, con una lista
<code>"default": null</code>	La pieza que faltaba en el paso 3. Le dice al lector nuevo qué asumir cuando el dato viejo no trae este campo

Pregúntale otra vez

Los mismos dos comandos del paso 3, con el archivo cambiado.

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"$(tr -d '\n' < infra/schemas/pedido-v2-compatible.avsc | sed
's/"/\\\"/g')\"}" \
  http://localhost:8081/compatibility/subjects/novatech.lab10.pedidos-
value/versions/latest
```

LO QUE VAS A VER

```
{"is_compatible":true}
HTTP 200
```

El mismo **HTTP 200** que devolvió el **false** del paso 3. Es la prueba de lo que se dijo allá, o sea que el código no es el veredicto.

Y se registra

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"$(tr -d '\\n' < infra/schemas/pedido-v2-compatible.avsc | sed
's/\\/\\\\/g')\\\"}\" \
  http://localhost:8081/subjects/novatech.lab10.pedidos-value/versions \
  | fold -s -w 88
```

LO QUE VAS A VER · EL SCHEMA DEL MEDIO VA RECORTADO

```
{"id":2,"version":2,"guid":"8039c9ea-b34f-edeb-4674-e9dd977a4b01","schemaType":"AVRO","s
chema":{"\"type\": \"record\", \"name\": \"Pedido\", ..., {\"name\": \"prioridad\",
\"type\": [\"null\", \"string\"], \"default\": null}}}}
HTTP 200
```

HTTP 200, no **409**. Es el mismo comando que hace dos minutos devolvió un conflicto, contra el mismo subject y la misma versión vigente. Lo único distinto es que este schema trae *default*.

Y la vieja no se borró

```
curl -s http://localhost:8081/subjects/novatech.lab10.pedidos-value/versions
```

LO QUE VAS A VER

```
[1,2]
```

LAS DOS SIGUEN AHÍ, Y TIENEN QUE SEGUIR

El mensaje que escribiste en el paso 4 lleva pegado el **id: 1**. Si la **v1** desapareciera, ese **mensaje quedaría ilegible para siempre**. El Registry no es un archivo de configuración que se sobrescribe. Es un registro histórico, y por eso vive en un tópico de Kafka llamado `_schemas` y no en un archivo.

VAS BIEN SI...

La consulta te dio `true` , el registro te dio `HTTP 200` con `version: 2` , y la lista de versiones dice `[1,2]` .

SI TE DEVUELVE `VERSION: 1` OTRA VEZ

Registraste el mismo schema dos veces. **El Registry no crea una versión nueva si el schema es idéntico a uno que ya tiene.** Te devuelve el que ya existía. Comprueba que el archivo es el `-v2-compatible` .

Quién puso esa regla

Qué vamos a hacer

El mensaje de error del paso 3 terminaba con `compatibility: 'BACKWARD'`. **Alguien eligió eso.** Vamos a buscar dónde está puesto, porque la respuesta sorprende y es la que importa en SUNAT.

Pregúntale al subject

```
curl -s http://localhost:8081/config/novatech.lab10.pedidos-value
```

LO QUE VAS A VER

```
{"error_code":40408,"message":"Subject 'novatech.lab10.pedidos-value' does not have subject-level compatibility configured"}
```

El subject no tiene modo propio. El `BACKWARD` con que te rechazó el paso 3 no estaba puesto ahí.

Está un nivel más arriba

```
curl -s http://localhost:8081/config
```

LO QUE VAS A VER

```
{"compatibilityLevel":"BACKWARD"}
```

Estaba puesto **en el clúster entero**. Todos los subjects que no digan lo contrario heredan esa regla.

POR QUÉ ESTO IMPORTA, Y MUCHO

Significa que **una sola persona, cambiando una sola línea, cambia las reglas de todos los tópicos a la vez.** Y que un tópico puede tener su propia regla sin que nadie lo note, porque el `/config` global no la muestra. **Cuando alguien te diga «nuestro Registry está en BACKWARD», la pregunta correcta es en qué nivel.**

Compruébalo poniéndole un modo propio

```
curl -s -X PUT -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data '{"compatibility":"FORWARD"}' \
  http://localhost:8081/config/novatech.lab10.pedidos-value

curl -s -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data "{\"schema\": \"$(tr -d '\\n' < infra/schemas/pedido-v3-incompatible.avsc | sed
's/\"/\\\\\"/g')\"}" \
  http://localhost:8081/compatibility/subjects/novatech.lab10.pedidos-
value/versions/latest
```

-X PUT — cambiar una configuración es un **PUT**, no un **POST**. Un **PUT** reemplaza lo que hubiera **/config/<subject>** — con el subject al final, pone la regla **solo en ese subject**. Sin el subject, la pondría en el clúster entero

LO QUE VAS A VER · LAS DOS RESPUESTAS SALEN PEGADAS

```
{"compatibility":"FORWARD"}{"is_compatible":true}
```

SALEN PEGADAS Y NO ES UN ERROR

El Registry no termina sus respuestas con un salto de línea. Como corriste dos `curl` seguidos, la segunda respuesta empieza donde acabó la primera. Es feo y es lo que imprime. Si te molesta, agrega `; echo` al final de cada uno.

Y ahora lo importante

El mismo schema que hace tres pasos era incompatible ahora es compatible. No cambió el schema. No cambió el contrato vigente. **Cambió la regla con la que se juzga.**

Modo	Qué exige	Cuándo se usa
BACKWARD	El lector nuevo lee datos viejos	El de fábrica. Primero se actualizan los consumidores
FORWARD	El lector viejo lee datos nuevos	Cuando hay consumidores que no puedes actualizar, como un <i>mainframe</i> o el sistema de un tercero
FULL	Las dos cosas a la vez	Máxima seguridad, mínima libertad para cambiar
NONE	Nada. Entra cualquier cosa	Solo desarrollo local. En producción es lo mismo que no tener Registry

Déjalo como estaba

```
curl -s -X PUT -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data '{"compatibility":"BACKWARD"}' \
  http://localhost:8081/config/novatech.lab10.pedidos-value
```

LO QUE VAS A VER

```
{"compatibility":"BACKWARD"}
```

OJO, ESTO NO LO DEVUELVE DEL TODO A COMO ESTABA

El subject ahora **tiene** una regla propia, que da la casualidad de que dice lo mismo que la global. **Antes no tenía ninguna.** Si vuelves a correr el `/config/<subject>` del principio de este paso, ya no te va a dar el `40408`. Para volver de verdad al estado inicial hay que borrar la regla del subject con un `DELETE`, o levantar el lab de nuevo.

LO LOGRASTE SI...

Viste el `40408` del subject, el `BACKWARD` del clúster, y el mismo schema del paso 3 devolviendo `is_compatible: true` bajo `FORWARD`.

Anota una respuesta. De los tópicos que tu equipo tiene hoy en producción, ¿cuántos tienen subject registrado, y quién decidió su modo de compatibilidad?

Lo que aprendiste

1 Un tópico sin subject registrado no tiene contrato.

No importa cuánta documentación exista. Si no está en el Registry, no lo aplica nadie. Lo comprobaste en el paso 1, con un clúster recién levantado que contestó una lista vacía.

2 Todo campo que se agregue lleva *default*.

Es la única diferencia entre el schema del paso 3 y el del paso 5, y es la diferencia entre un despliegue que sale y uno que no. Y la comprobación cuesta un segundo, con el `/compatibility` que pregunta sin escribir. Eso va en el *pipeline*, no en la consola.

3 No hay puerta trasera, y el que rechaza no es el broker.

El contrato lo aplica la biblioteca del cliente, antes de que el mensaje salga a la red. Por eso el error dice `SerializationException` y el tópico se queda en cero. El broker nunca se enteró de que alguien intentó escribir.

4 El modo de compatibilidad es una decisión, y las versiones viejas no se borran.

El propio mensaje de error dice `compatibility: 'BACKWARD'`, y el paso 6 mostró que estaba puesto en el clúster entero y no en el tópico. En SUNAT ese alguien tiene que tener nombre. Y cada mensaje escrito lleva el `id` de su schema, así que borrar una versión es dejar ilegibles los datos que la usaron.

Para profundizar

A · Producir y consumir Avro de verdad

El paso 4 escribió un mensaje con un comando crudo, para poder romperlo a propósito. Los envoltorios del curso hacen lo mismo, más corto.

```
kafka-cli/produce-pedido-avro.sh 7 1001 "Caja premium" 10 25000.00 pendiente
kafka-cli/consume-avro.sh novatech.lab10.pedidos
```

Ctrl+C para salir del consumidor, que no termina solo.

LO QUE SALE DEL CONSUMIDOR · LAS DOS PRIMERAS LÍNEAS

```
{"id":7,"cliente_id":1001,"producto":"Caja
premium","cantidad":10,"monto":25000.0,"estado":"pendiente"}
{"id":1,"cliente_id":1001,"producto":"Caja
premium","cantidad":10,"monto":25000.0,"estado":"pendiente"}
```

Lo que hay que mirar. El consumidor imprime JSON, pero **en el tópic no hay JSON**. Hay bytes binarios con el `id` del schema adelante. Quien traduce es el consumidor, que va al Registry con ese número y baja el contrato.

El productor imprime además un aviso que sale siempre, `Warning: --property is deprecated and will be removed in a future version`. **Es del envoltorio, no tuyo**, y no afecta a nada.

La pregunta que vale. ¿Qué pasa si apagas el contenedor `schema-registry` con `docker stop schema-registry` y vuelves a consumir?

B · La carga masiva y el contrato de la clave

```
kafka-cli/produce-flood-pedidos.sh 50
kafka-cli/produce-clientes-seed.sh
curl -s http://localhost:8081/subjects
```

LO QUE SALE

```
["novatech.lab10.clientes-key","novatech.lab10.clientes-value","novatech.lab10.pedidos-value"]
```

Lo que hay que mirar. Después de los clientes hay **tres** subjects, no dos. Apareció un `-key`.

El *seed* de clientes escribe con clave, y **la clave tiene su propio contrato, independiente del valor**. Es la convención `TopicNameStrategy` funcionando en las dos mitades del mensaje.

C · El tópico donde vive el talonario

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --topic _schemas
```

LO QUE SALE

```
Topic: _schemas TopicId: lrTNQ3gSQgy--jmkd7B6yw PartitionCount: 1      ReplicationFactor: 3  
Configs: min.insync.replicas=2,cleanup.policy=compact  
      Topic: _schemas Partition: 0      Leader: 3      Replicas: 3,1,2 Isr: 3,1,2      Elr:  
LastKnownElr:
```

Lo que hay que mirar. Tres cosas, y las tres son decisiones de diseño.

Una sola partición. El orden de los cambios de contrato importa, y una sola partición es la única forma de garantizar orden total en Kafka.

`cleanup.policy=compact`, que es la compactación del laboratorio 05. Kafka conserva el último valor de cada clave para siempre, en vez de borrar por antigüedad. **Un contrato no puede caducar a los siete días.**

Factor de réplica 3. Si se pierde este tópico, todos los datos Avro del clúster quedan ilegibles.

D · Los envoltorios de `schema-cli/`, y por qué esta guía no los usa

Hacen lo mismo que los `curl` del recorrido, en una línea.

```
schema-cli/list-subjects.sh  
schema-cli/register-schema.sh novatech.lab10.pedidos-value infra/schemas/pedido.avsc  
schema-cli/check-compatibility.sh novatech.lab10.pedidos-value infra/schemas/pedido-v3-  
incompatible.avsc
```

Y los cuatro dependen de `python3`, que Git Bash para Windows no trae y que ningún README declara. Compruébalo:

```
grep -n python3 schema-cli/*.sh
```

LO QUE PASA SIN `PYTHON3`, MEDIDO, Y POR QUÉ UNO ES PEOR QUE EL OTRO

`register-schema.sh` falla de frente, y eso está bien:

```
schema-cli/register-schema.sh: line 17: python3: command not found
```

`list-subjects.sh` falla en silencio, y eso es una trampa:

```
[List Subjects]
```

Imprime su encabezado y **nada más**. Ni un error. La razón es que el mensaje de `command not found` cae dentro del `>/dev/null` del final de la línea. **Un alumno en Windows leería eso como «no hay ningún subject registrado», que es una respuesta falsa y no un fallo.** Por eso el recorrido va por `curl`.

E · El desafío del contrato propio

Escribe tú un `.avsc` que agregue dos campos al pedido, uno compatible y otro no, y predice el veredicto de cada uno **antes** de preguntarle al Registry. Documenta en el reporte qué predijiste, qué contestó y por qué.

Es el ejercicio que convierte la regla del *default* en algo que sabes aplicar y no solo repetir.

F · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre las actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh    # ver el estado del laboratorio
bin/stop-lab.sh       # apagar los contenedores
```

`bin/stop-lab.sh` solo detiene los contenedores y no borra nada.

Ojo al reanudar. `bin/start-lab.sh` reconstruye el laboratorio desde cero, así que **los schemas que registraste no sobreviven**, porque se van con el tópico `_schemas`. Si escribiste un contrato propio en el bloque E, guárdate el archivo antes.

LO QUE DICE EL VALIDADOR SI TODO ESTÁ EN ORDEN

Validador del Lab 11 – estado actual

- ✓ los 3 brokers están healthy
 - ✓ Schema Registry responde en :8081/subjects
 - ✓ el subject novatech.lab10.pedidos-value está registrado
- ✓ LAB EN BUEN ESTADO (3 verificaciones OK)

LO QUE TE LLEVAS

El desarrollador del martes no era descuidado. Hizo el cambio más inocente que existe, lo probó, y funcionó. **Lo que le faltaba era una compuerta que le dijera que no el martes**, en su pantalla, con el nombre del campo.

Eso es todo lo que compra Schema Registry, y es muchísimo. **Adelantar el error al momento y a la persona que lo puede arreglar.**